Phantom AI Code Reviewer: Architecture, Workflow & Interview Guide

Table of Contents

1. Project Introduction & Impact

Overview: Phantom AI Code Reviewer is a production-grade, event-driven SaaS application that automates GitHub Pull Request (PR) reviews. It acts as a virtual "Principal QA Engineer" by analyzing code diffs across four distinct dimensions: Logic, Syntax, Performance, and Security.

The Impact: * **Zero-Latency Reviews:** Delivers comprehensive PR feedback in under 1 second using parallel asynchronous execution. * **Token & Cost Optimization:** Bypasses stringent LLM API rate limits using a custom Round-Robin key rotation algorithm. * **Developer Productivity:** Eliminates manual review bottlenecks, catching critical vulnerabilities (e.g., hardcoded secrets, $O(N^2)$ loops) before code reaches production.

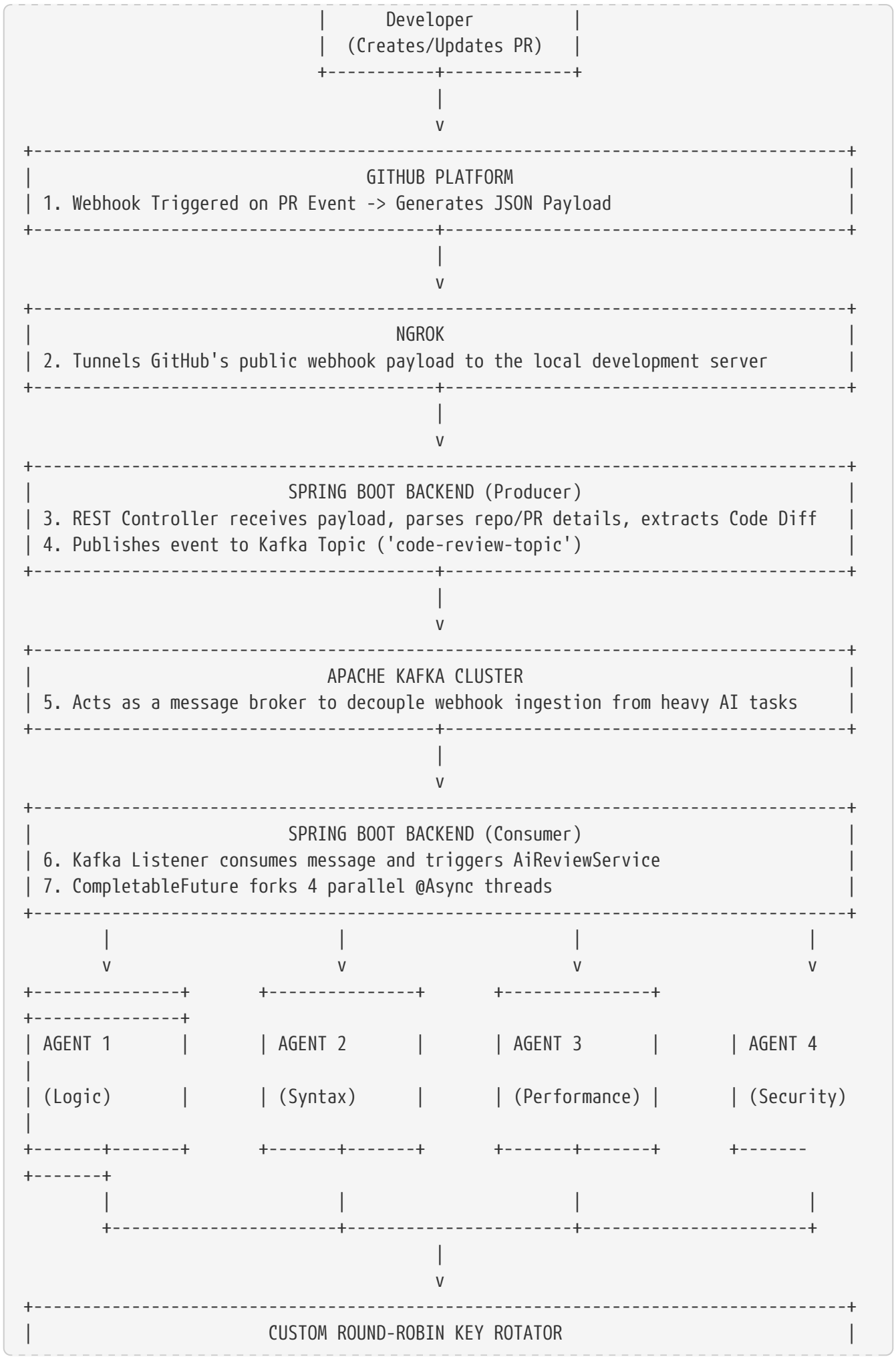
Core Philosophy (What I Learned): Building this project transitioned my mindset from writing "scripts that work" to "systems that scale." I learned the importance of Separation of Concerns (Multi-Agent LLMs instead of one massive prompt), Event-Driven Architecture (Kafka), and Thread Safety in high-concurrency environments.

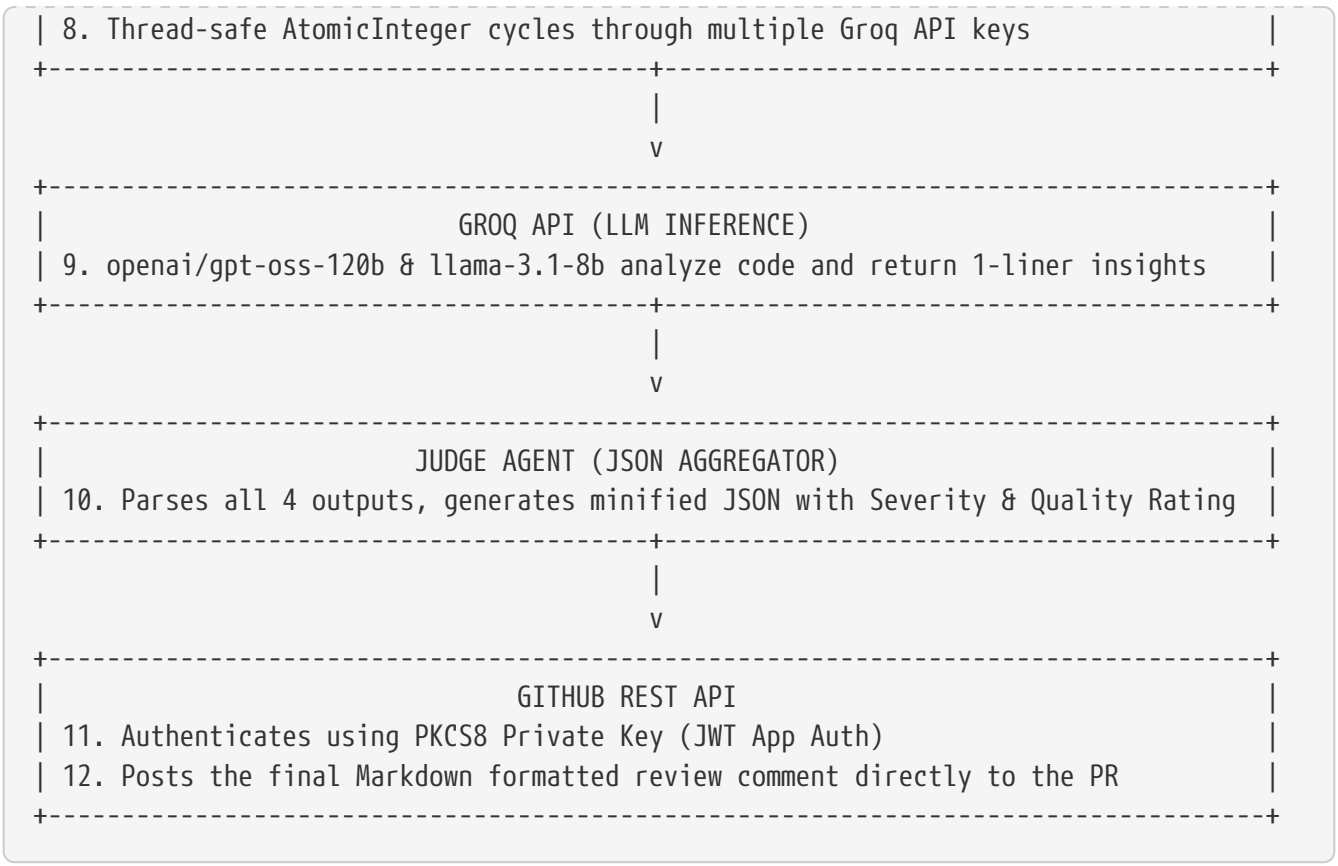
---

2. Architecture & Complete Workflow

The system is built on an event-driven, decoupled architecture.

```
                                    +------------------------+
```


Developer
(Creates/Updates PR)
GITHUB PLATFORM
1. Webhook Triggered on PR Event -> Generates JSON Payload
NGROK
2. Tunnels GitHub's public webhook payload to the local development server
SPRING BOOT BACKEND (Producer)
3. REST Controller receives payload, parses repo/PR details, extracts Code Diff
4. Publishes event to Kafka Topic ('code-review-topic')
APACHE KAFKA CLUSTER
5. Acts as a message broker to decouple webhook ingestion from heavy AI tasks
SPRING BOOT BACKEND (Consumer)
6. Kafka Listener consumes message and triggers AiReviewService
7. CompletableFuture forks 4 parallel @Async threads
AGENT 1
(Logic)
AGENT 2
(Syntax)
AGENT 3
(Performance)
AGENT 4
(Security)
CUSTOM ROUND-ROBIN KEY ROTATOR



3. Core Technologies & Key Learnings

- **Java 21 & Spring Boot:** Learned how to structure enterprise applications, manage dependency injection, and utilize `@Async` for non-blocking operations.
- **Apache Kafka:** Learned how to implement Pub/Sub messaging. Realized that without Kafka, a slow LLM API response would cause the GitHub Webhook to timeout and fail.
- **CompletableFuture (Concurrency):** Mastered parallel programming. Instead of waiting 4 seconds for 4 agents (1s each sequentially), the system runs them concurrently, returning results in ~780ms.
- **GitHub Apps & JWT Auth:** Learned the complexities of server-to-server authentication. Generating JWTs dynamically using cryptographic keys (`.pem` to `PKCS8`).
- **Groq API & LLM Prompt Engineering:** Learned that AI is unpredictable. Mastered the art of "Strict Prompting" to force LLMs to return minified JSON without markdown wrappers, saving tokens and preventing parsing crashes.

4. Challenges Faced & Engineered Solutions

4.1. Challenge 1: GitHub API 401 Unauthorized Errors

- **Problem:** GitHub kept rejecting my authentication tokens when trying to post comments. The

Java application couldn't read the standard RSA `.pem` key provided by GitHub.

- **Solution:** I researched Java's cryptographic standards and discovered `java.security` requires the PKCS#8 format. I used an OpenSSL command line utility to translate the key, and also debugged a critical edge-case where a single typo in the `App ID` (missing the last digit) was causing stealth auth failures.

4.2. Challenge 2: Groq API Rate Limiting (Token Exhaustion)

- **Problem:** Moving to a Multi-Agent architecture meant sending 4 heavy prompts simultaneously. This immediately spiked the "Tokens Per Minute" (TPM) limit, causing `429 Too Many Requests` errors.
- **Solution:** Instead of downgrading the architecture, I engineered a **Round-Robin API Key Rotator**. I utilized `java.util.concurrent.atomic.AtomicInteger` to maintain thread-safe increments across concurrent agent threads.

```
String currentKey = groqApiKeys[Math.abs(keyIndex.getAndIncrement() % groqApiKeys
.length)];
```

This perfectly distributed the load across multiple API limits without race conditions.

4.3. Challenge 3: JSON Parsing Crashes (`Unrecognized token`)

- **Problem:** The 5th "Judge Agent" was tasked with returning JSON. However, the LLM would occasionally wrap the response in markdown blocks (`json . . .`) or add preamble text, which caused Jackson's `ObjectMapper` to throw runtime exceptions.
- **Solution:** I applied defensive programming. I engineered a highly restrictive prompt ("CRITICAL INSTRUCTIONS: Output ONLY a raw, minified JSON object. ZERO preamble"), and added a robust string sanitizer (`.replace("```json", "")`) before passing it to Jackson's `readTree()`.

4.4. Challenge 4: LLM Context Window Exhaustion & High API Costs

- **Problem:** Sending an entire codebase or even full files for every single PR commit would immediately exceed the LLM's context window (token limit) and skyrocket API costs.
- **Solution:** Engineered a **Parallel Diff Analysis** approach. Instead of fetching the repo, the system leverages GitHub's `vnd.github.v3.diff` header to extract **only** the specific lines that were added or modified. This creates a highly optimized micro-payload. Multiplexing this small diff across 4 parallel agents cuts token consumption by over 90% while maintaining deep, specialized analysis.

5. Rapid Revision: Interview Q&A

Q1: Why did you use Apache Kafka for this project instead of standard REST calls?

Answer: GitHub webhooks have strict timeout limits (usually 10 seconds). If the AI takes 12 seconds to review a large PR, GitHub drops the connection and marks the webhook as failed. Kafka acts as a shock absorber. The REST controller receives the payload, instantly puts it in a Kafka topic, and returns a `200 OK` to GitHub in milliseconds. The heavy AI processing happens asynchronously in the background via the Kafka Consumer.

Q2: Explain how your Multi-Agent concurrency works. What happens if one agent fails?

Answer: I used Java's `CompletableFuture` combined with Spring's `@Async`. I fork four separate asynchronous tasks for Logic, Syntax, Performance, and Security. Then I use `CompletableFuture.allOf().join()` to wait for all of them to complete simultaneously. If one agent fails (e.g., API error), my try-catch block inside the agent's service catches it and returns a fallback string ("Analysis skipped due to API error"), ensuring the other three agents' results are still successfully merged and posted.

Q3: You mentioned a "Round-Robin Key Rotator". Why did you need `AtomicInteger` instead of a standard `int`?

Answer: Because the four AI agents run in parallel on separate threads. A standard `int++` operation is not atomic (it involves read, increment, write). If two threads read the integer at the exact same millisecond, they might both get the same API key, defeating the purpose and causing rate limits. `AtomicInteger.getAndIncrement()` uses CAS (Compare-And-Swap) at the CPU level, ensuring absolute thread safety without the performance bottleneck of `synchronized` blocks.

Q4: Why divide the AI into 4 Agents instead of asking one model to do everything in one prompt?

Answer:

Three reasons:

1. **Accuracy:** LLMs suffer from "attention degradation" with massive prompts. Focusing an agent purely on "Security" yields deeper, more aggressive audits.
2. **Speed:** 4 targeted prompts running in parallel are faster than 1 massive prompt executing sequentially.
3. **Cost/Routing Optimization:** In the future, I can route the "Syntax" task to a cheaper, faster 8B model, while keeping the "Logic" task on a highly capable 120B model, drastically optimizing API costs.

Q5: How did you extract the precise code changes from the GitHub Webhook?

Answer: The initial webhook payload doesn't contain the actual code; it only contains metadata about the PR. I had to make an authenticated callback to the GitHub REST API using the PR's `diff_url`. I implemented an HTTP client with specific `Accept: application/vnd.github.v3.diff`

headers to pull the raw, line-by-line git diff for the AI to analyze.